

3x6 martices Practical- 5

Aim: To Implementation of a knapsack problem using dynamic programming.

Knapsack

Algorithm:

Prac - 5

Input :-

- * Number of items n
- * weight $w[i]$ and value $v[i]$ of each item.
- * Knapsack capacity w

output :-

- * max value that can be obtained
- * selected items.

step-1 :- start

step-2 :- Read the no. of item n and knapsack capacity w .

step-3 :- Read the weight $w[i]$ and value $v[i]$ of each item.

step-4 :- initialize the final row and first column of DP table to 0.

step-5 :- for each item $i=1$ to n , repeat step 6.

step-6 :- for each capacity $j=1$ to w :

if $w[i] \leq j$, calculate:

$$dp[i][j] = \max\{v[i] + dp[i-1][j-w[i]], dp[i-1][j]\}$$

otherwise,

$$dp[i][j] = dp[i-1][j]$$

step-7 :- The max value is stored in $dp[n][w]$.

step-8 :- Trace the dp table from $dp[n][w]$ but w to find the selected items

step-9 :- Display the dp table, max value, and selected item.

step-10 :- stop.



Program:

```
#include<iostream>

using namespace std;

class Knapsack {
private:
    int w[10];
    int v[10];
    int n;
    int W;
    int dp[10][100];
public:
    void getInput() {
        cout << "Enter number of items : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }
    void solve() {
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {
                if(w[i] <= j) {
                    dp[i][j] = max(
                        v[i] + dp[i-1][j-w[i]],
                        dp[i-1][j]
                    );
                } else {
                    dp[i][j] = dp[i-1][j];
                }
            }
        }
    }
};
```



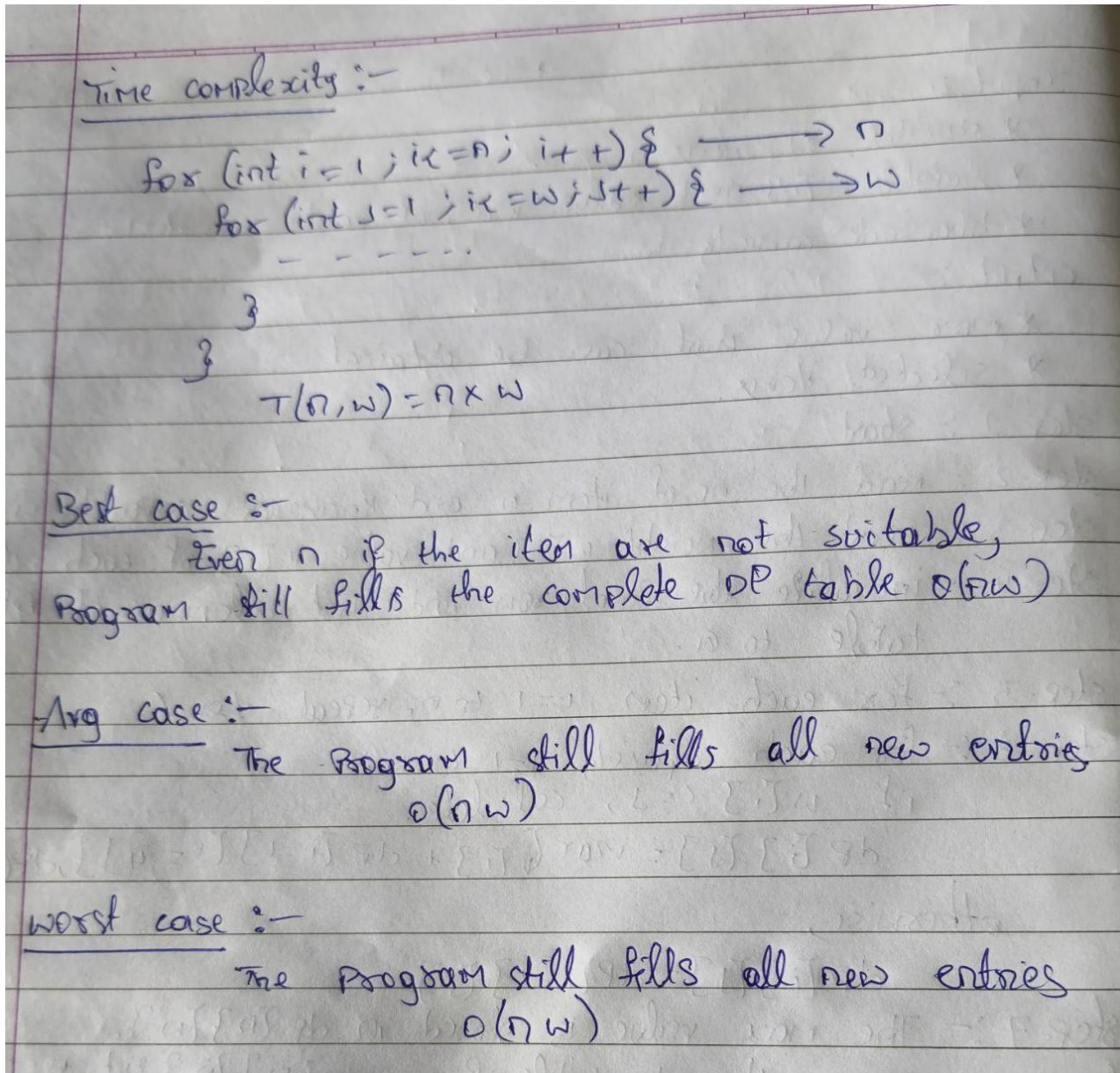
```
    }  
    }  
}  
  
void printTable() {  
    cout << "\nDP Table:" << endl;  
    cout << "    ";  
    for(int j = 0; j <= W; j++)  
        cout << j << "    ";  
    cout << endl;  
    for(int i = 0; i <= n; i++) {  
        cout << "i=" << i << "    ";  
        for(int j = 0; j <= W; j++)  
            cout << dp[i][j] << "    ";  
        cout << endl;  
    }  
}  
  
void findItems() {  
    cout << "\nSelected Items:" << endl;  
    int j = W;  
    for(int i = n; i >= 1; i--) {  
        if(dp[i][j] != dp[i-1][j]) {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") TAKEN " << endl;  
            j = j - w[i];  
        } else {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") NOT taken " << endl;  
        }  
    }  
}
```

```
}  
}  
void showResult() {  
    cout << "\nMaximum Value = "  
        << dp[n][W] << endl;  
}  
};  
int main() {  
    Knapsack k;  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
    return 0;  
}
```

Output:

```
● PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-5\knapsack.exe"  
Enter number of items : 4  
Enter knapsack capacity : 8  
Item 1 weight and value : 2  
1  
Item 2 weight and value : 3  
2  
Item 3 weight and value : 4  
5  
Item 4 weight and value : 5  
6  
  
DP Table:  
  0  1  2  3  4  5  6  7  8  
i=0  0  0  0  0  0  0  0  0  0  
i=1  0  0  1  1  1  1  1  1  1  
i=2  0  0  1  2  2  3  3  3  3  
i=3  0  0  1  2  5  5  6  7  7  
i=4  0  0  1  2  5  6  6  7  8  
  
Maximum Value = 8  
  
Selected Items:  
Item 4 (w=5, v=6) TAKEN  
Item 3 (w=4, v=5) NOT taken  
Item 2 (w=3, v=2) TAKEN  
Item 1 (w=2, v=1) NOT taken  
○ PS C:\MinGW\bin> █
```


Time complexity:



Conclusion: The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the maximum value that can be obtained within the given knapsack capacity by considering whether each item should be selected or not selected. The time complexity of the algorithm is $O(nW)$ and the space complexity is $O(nW)$.